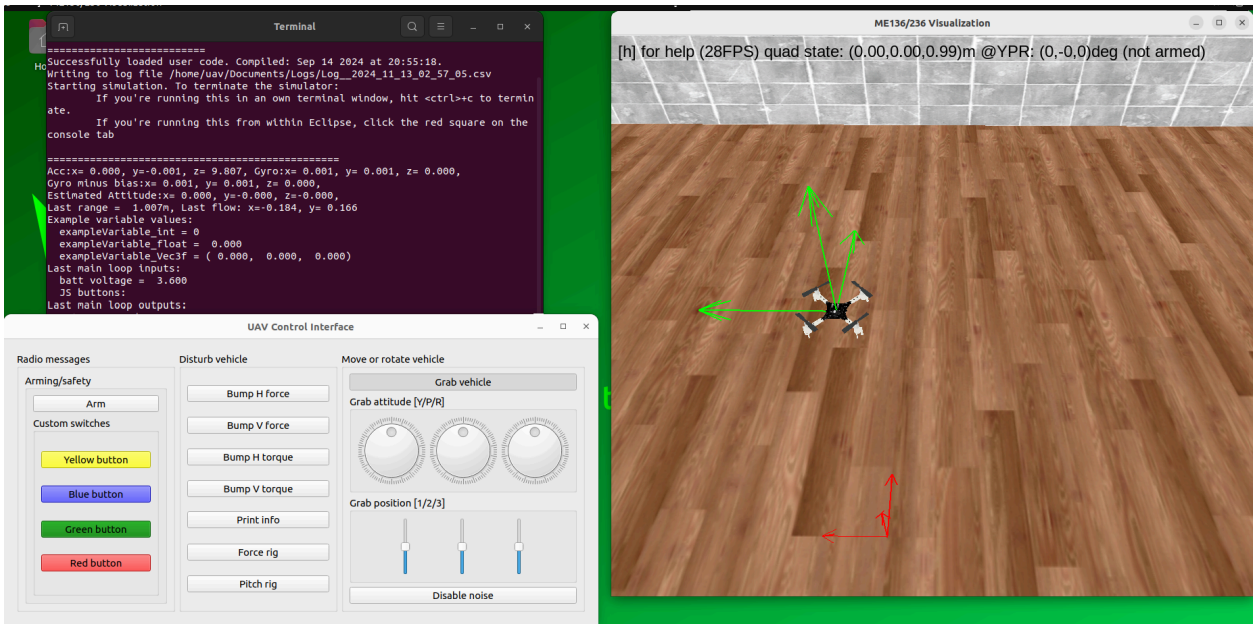
Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

Me 136 Lab 5 Report

1. Contributions:

Noah Koch: Plotting, interpreting plots, Extracting data, calculations, control system testing and constant selection.
Jason Khatkar: Coding, simulating, answering all the conceptual questions, plotting and labeling part 6
Ryan Montis: Interpreting data, labeling plots, conceptual evaluation.

2. Describe your experimental set-up (with photographs as necessary). [5%]

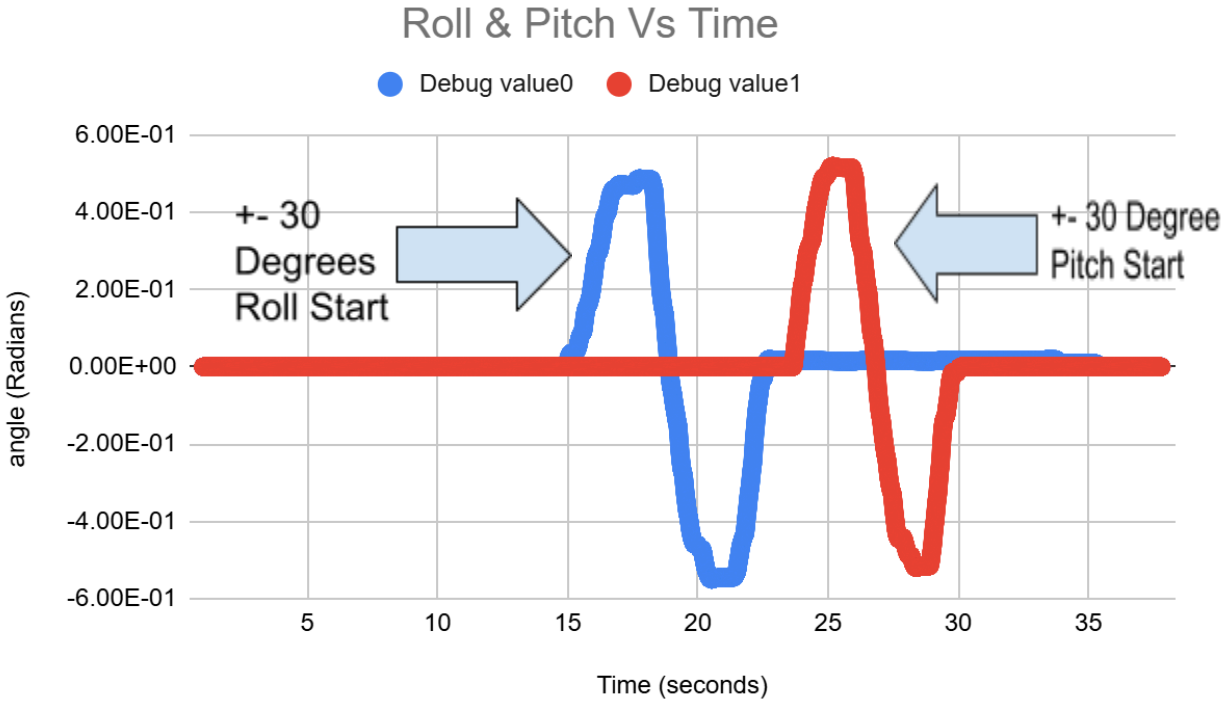


The quadcopter is placed on the ground, at rest and turned off. Once activated, the quadcopter rises into the air and begins to hover around the room, free to collide with the walls at some point.

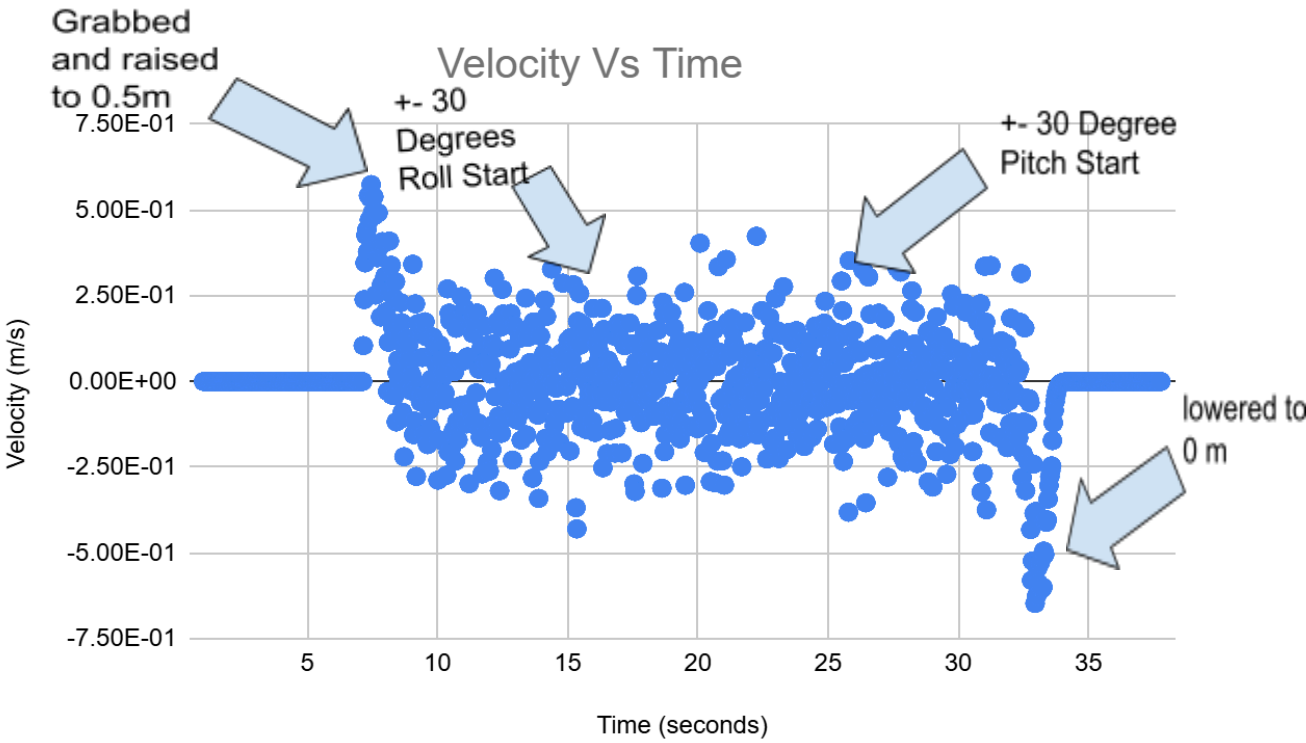
3.

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

i. The roll and pitch estimates

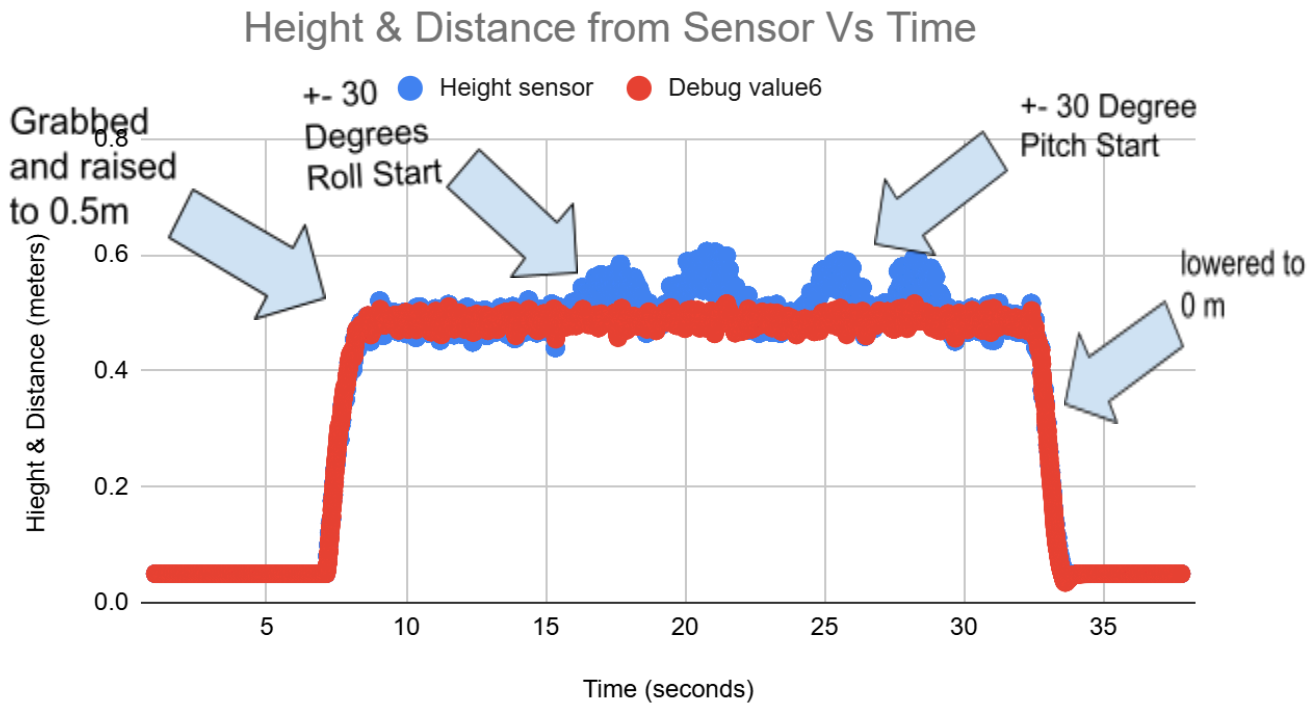


ii. The vertical velocity



Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

iii. The height estimate, and the measurements from the distance sensor.



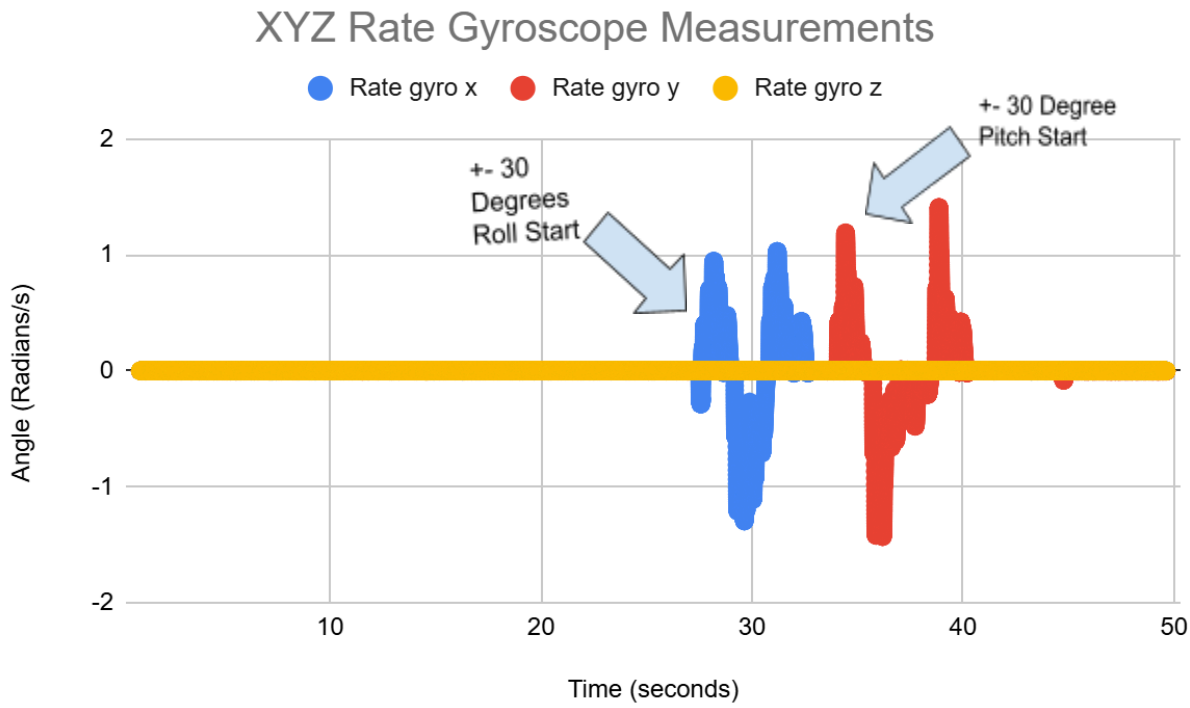
(c)

The estimator accurately tracks the transient motion of the quadcopter and more consistently tracks the actual hovering height value that the sensor data

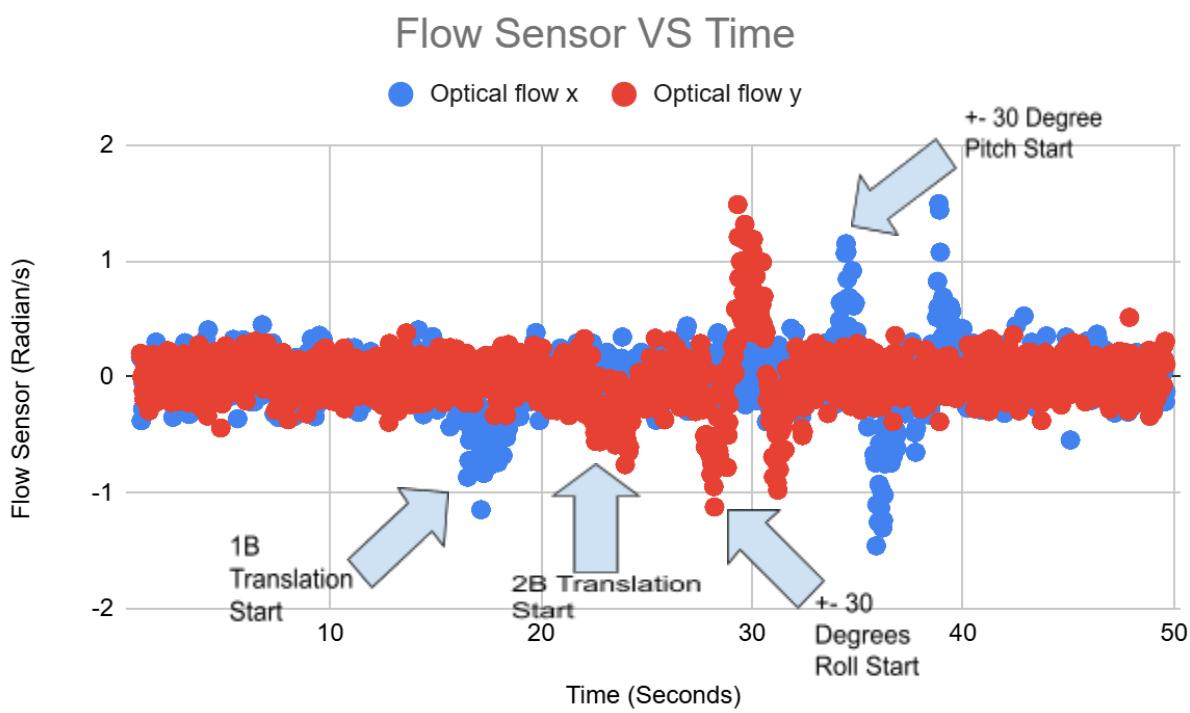
4.

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

i. The rate gyroscope measurements

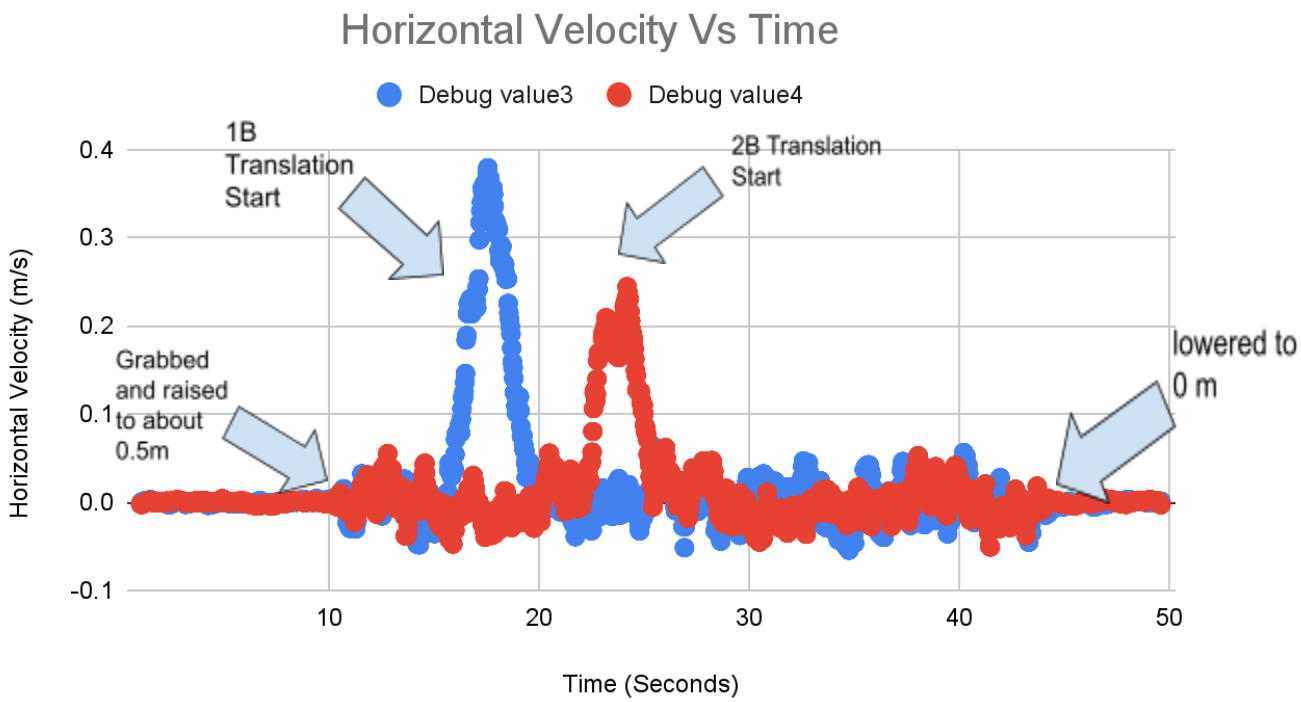


ii. The output from the flow sensor

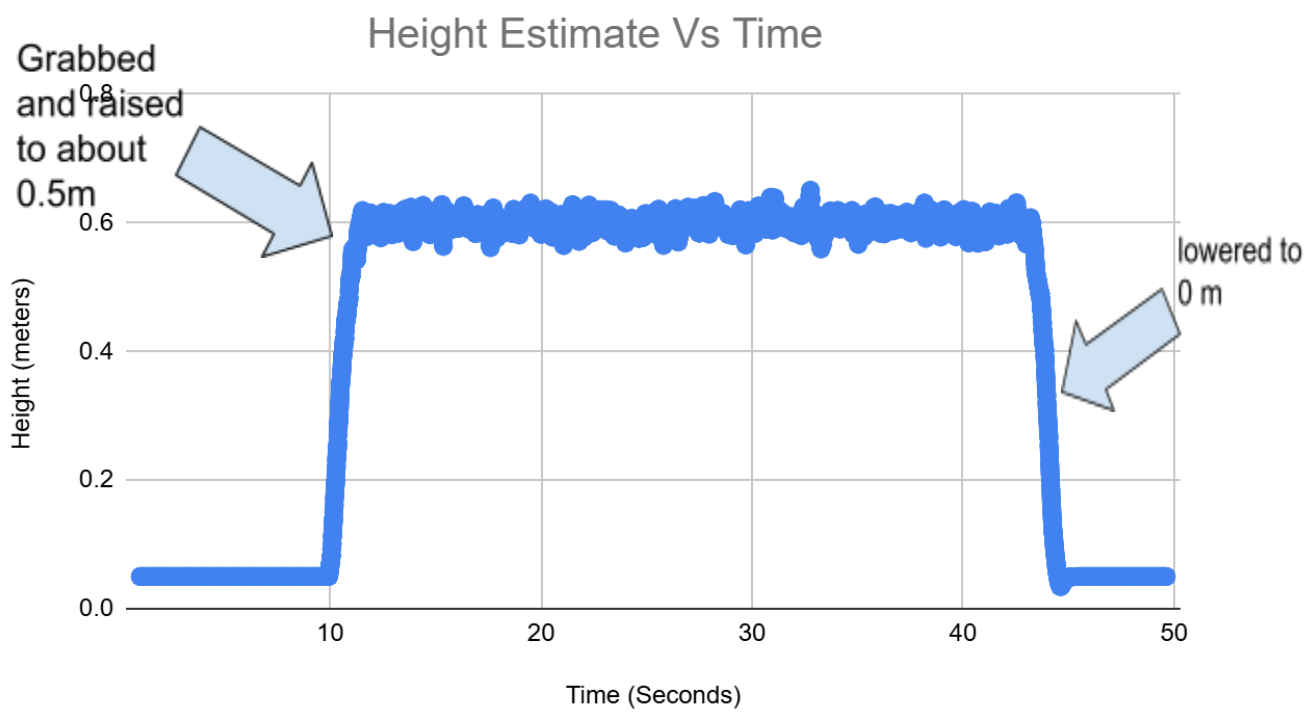


Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

iii. The estimated horizontal velocity



iv. The height estimate



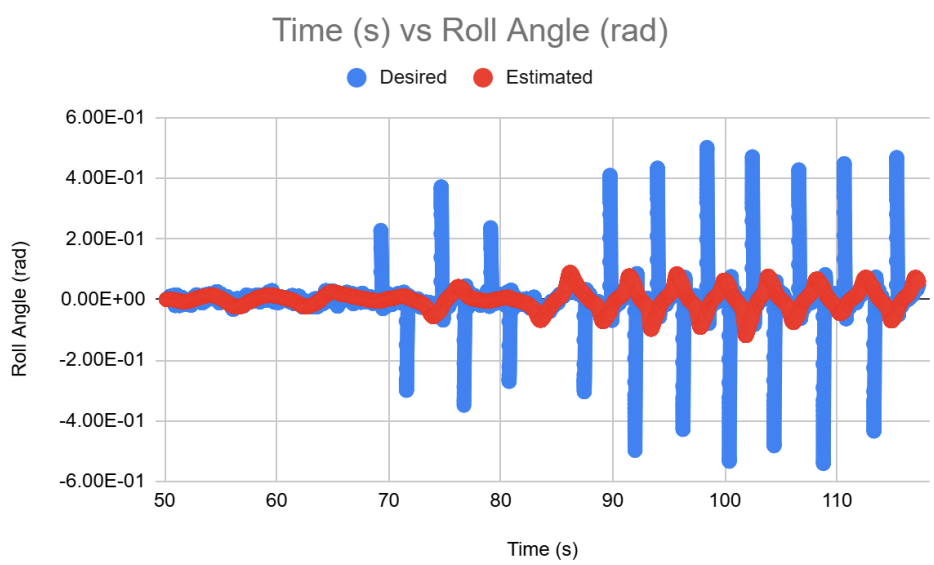
Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

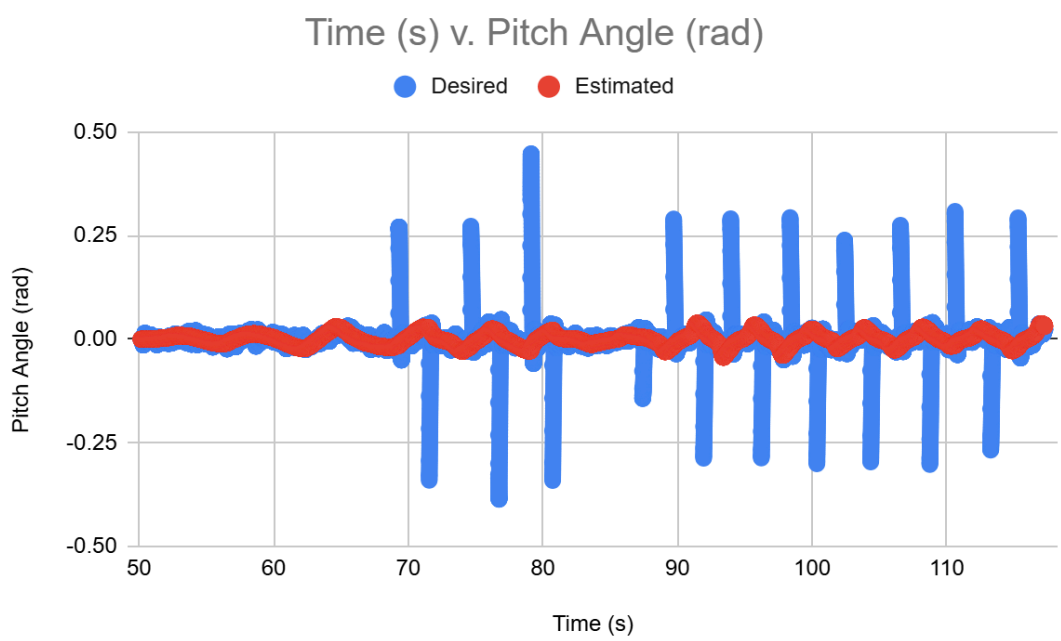
(c)

Each velocity estimator accurately spiked during the movement in that direction before returning to zero and the height sensor accurately oscillated about 0.5m and 0m as expected. These were much smoother and more readable than the direct sensor data in the plots above them.

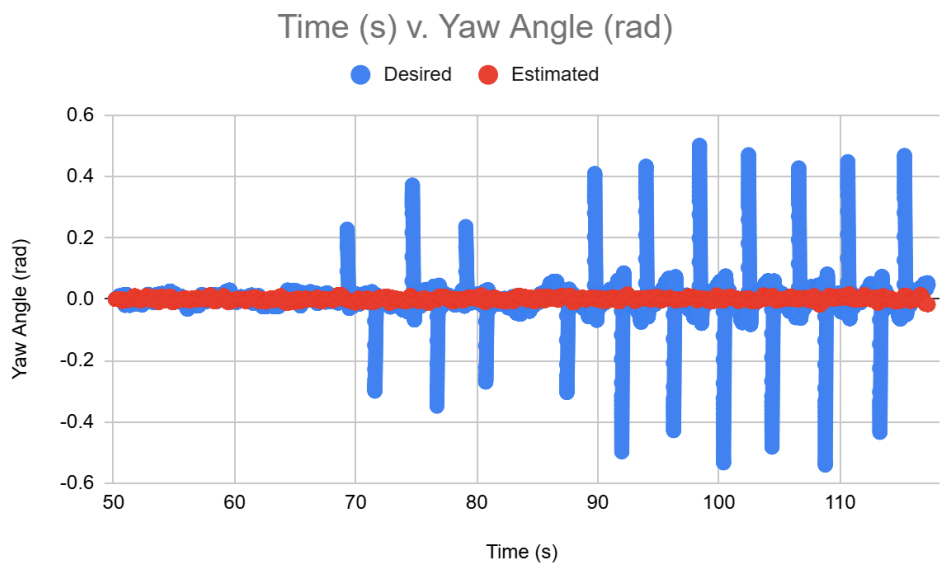
5.

i. The estimated angles, as well as the desired angles

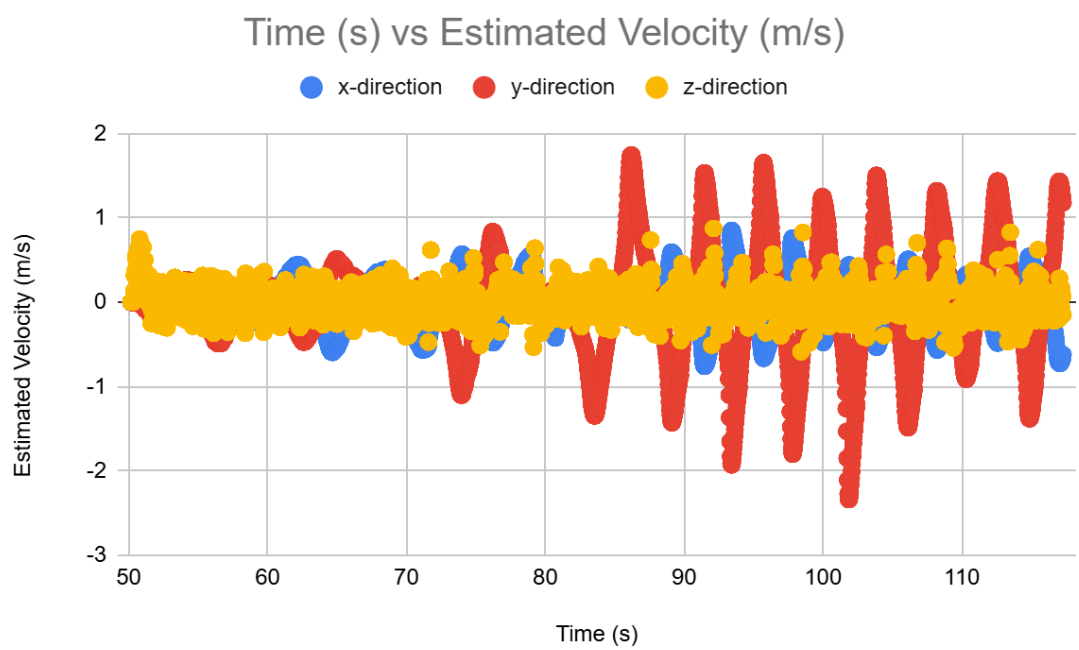




Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

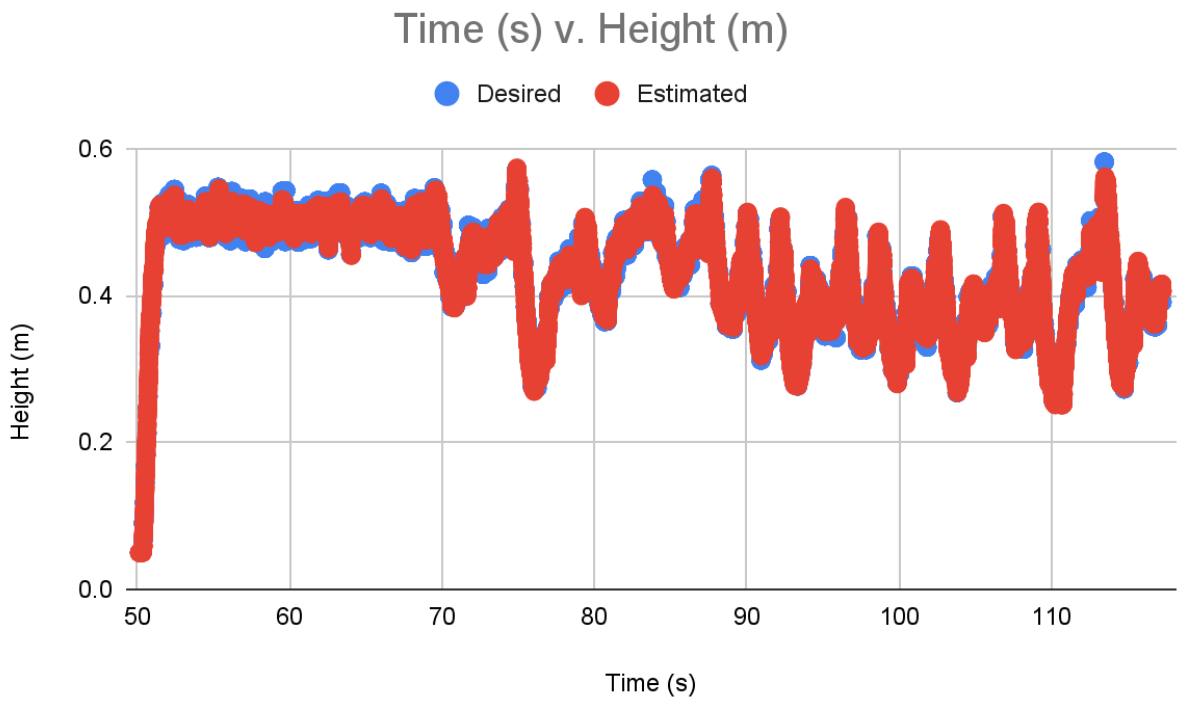


ii. The estimated vehicle velocity

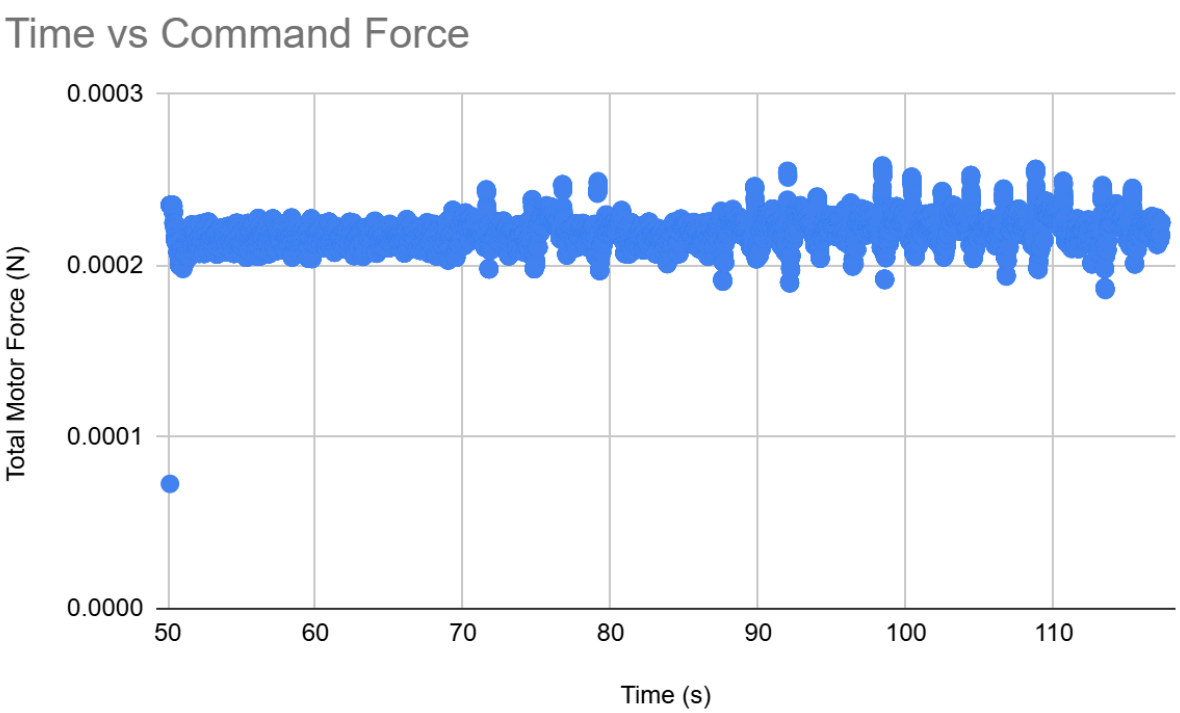


iii. The estimated height and the desired height

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024



iv. The total motor force (you'll have to implement the functions to transform the commands you receive over telemetry to [N], as you did in the C++ code).



(c) Comment on the flight performance.

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

Our quadcopter remained relatively stable for about 20 seconds, after which the oscillations became too extreme and it began to hit into walls. The motion noticeably drifted away from the origin within a few seconds but continually circled the origin

(d) Clearly explain any changes you made to the “standard” control layout, as presented in the labs so far, to improve performance.

The main change we made to the control layout is adding position control. We integrated the velocity estimate to make a position estimate and using it along with the velocity estimate in order to get a desired acceleration, which could then continue down the same control structure.

6. For your closed-loop hover flight analysis and modifications, do the below. Note that your hover performance will be evaluated experimentally (during the competition) in Lab 6, the description for which is available online.Here, we want to see your reasoning behind the modifications you make, and how you tested their efficacy. [30%]

(a) Explain what modifications you made to the standard control/estimation architecture, and what its effects were.

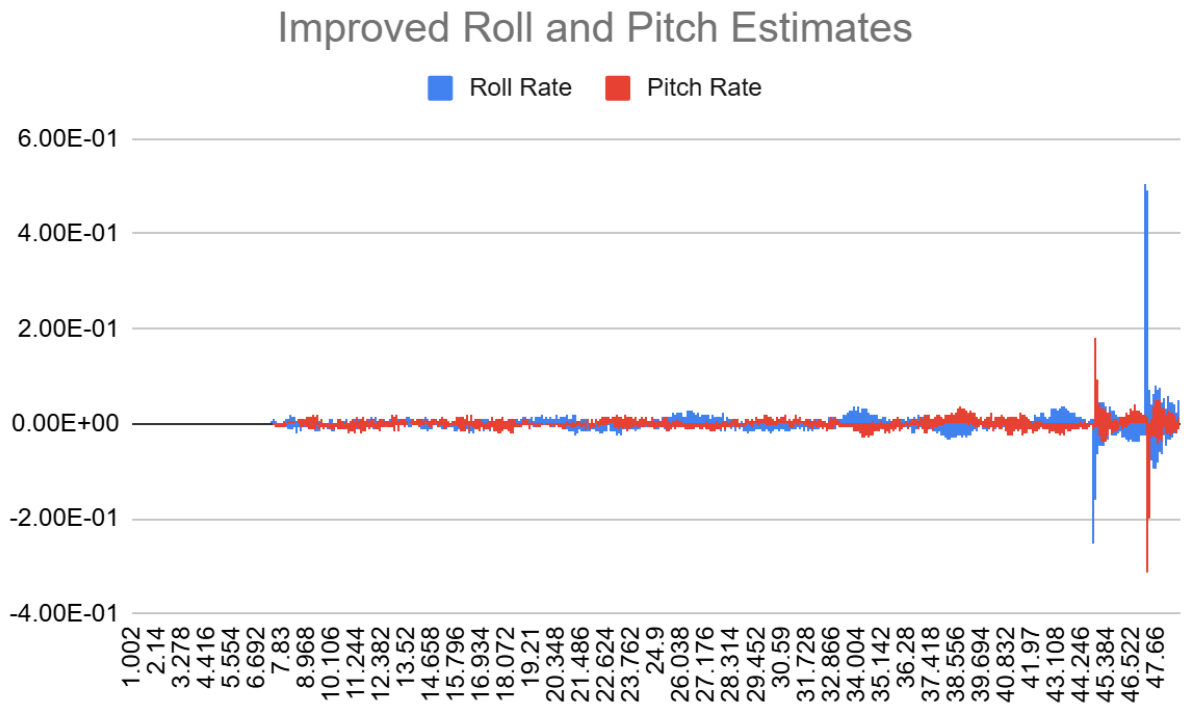
We modified the values for the time constants attached to roll rate, pitch rate, and yaw rate. We then added some position control to improve further and allow for a new set point for our following lab. We finally changed our vertical and horizontal velocity constants to critically damp the motion.

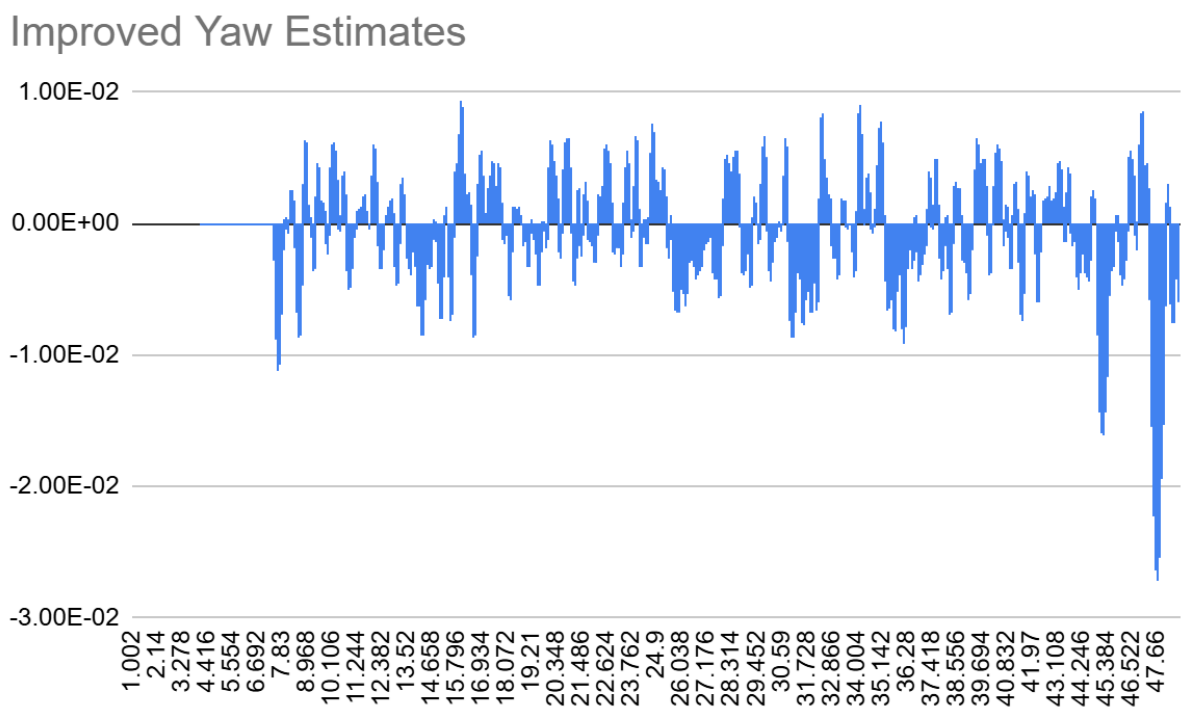
(b) Comment on the performance you could achieve: what do you believe is the main limitation that you face, to improve the performance?

The noise being produced by the sensors is the largest reason we couldn’t raise our performance any higher. We managed to improve the time before collision up to 72 seconds. In terms of estimation, our attitude estimation was the most accurate and we believe our optimization was the most efficient in that sector. The optical flow sensor and positional uncertainty was the largest drivers in the instability that we hit that didn’t let us improve the performance any further.

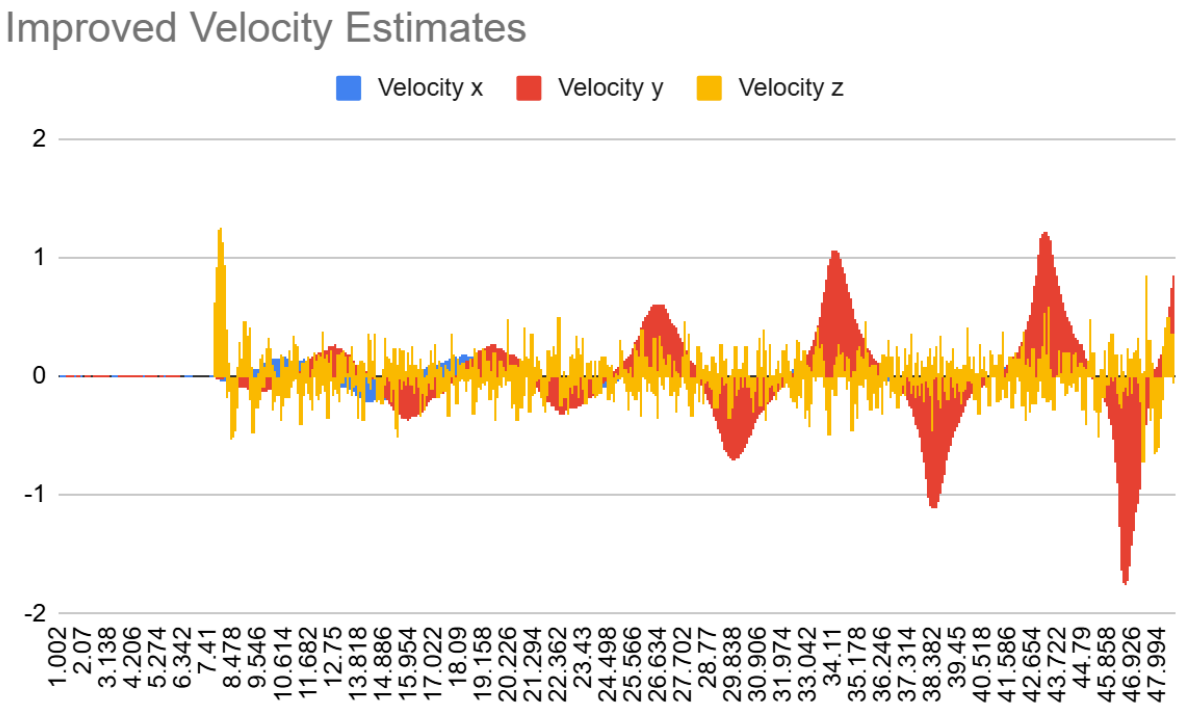
(c) Substantiate the above with clear experimental data.

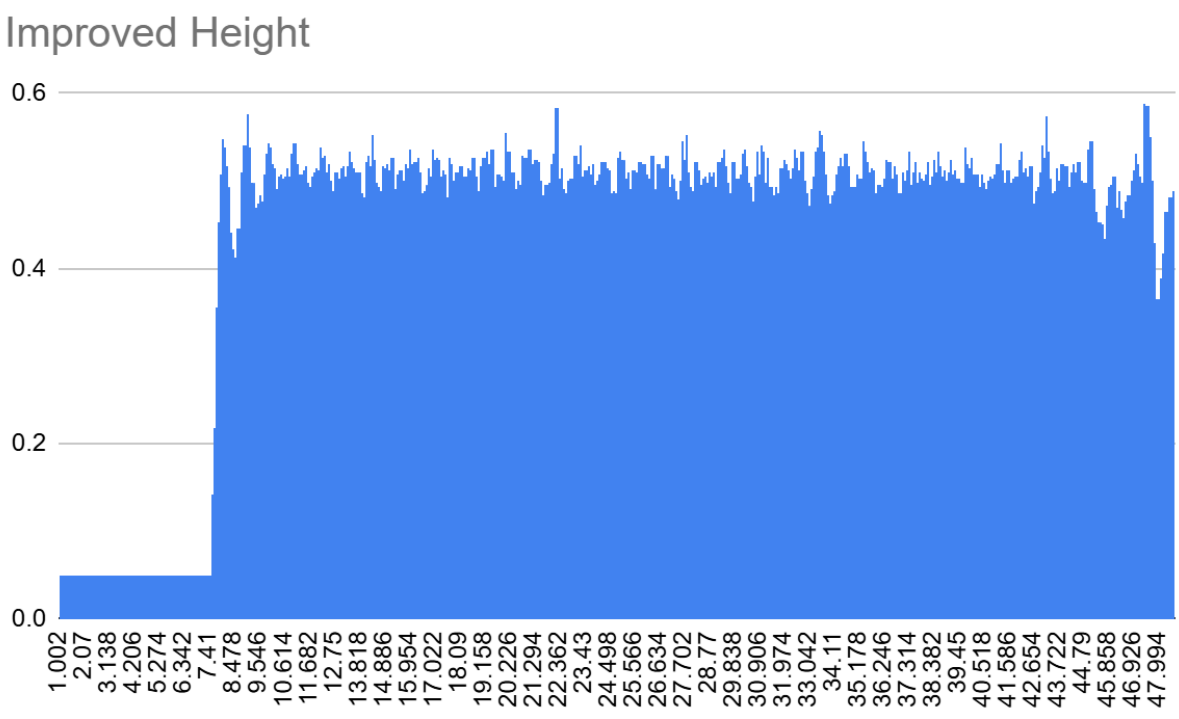
Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024





Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024





Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

Improvement Attempts

What we changed (variable and range)	Expected Improvement	Resulting Change	Worth keeping? And why	Optimal Value (and export a log for it)
Position Control	Little to no oscillation for much longer time periods	Decreased the oscillation at small displacements	Yes, allows for trends into different directions	
roll_rate, pitch_rate = 0.01-0.015	A decrease = less effect of noise	Took longer to hit wall (about 27 s), low values had bouncing off ground at initialization	Yes, in the range of 0.015~0.2 quadcopter stability performance doubled compared to 0.04	0.015
Yaw_rate = 0.01-0.065	Same as lowering roll and pitch	Took longer to hit wall (about 44 s), too low had it starting to get off ground then slowly going back down	Yes, continues to increase stability performance	0.065
Roll_angle,pitch_angle !=0.12, Yaw_angle != 0.2	Lower would increase performance /less noise	Decrease in performance in both directions	Original Values seems optimal, angle might be very dependent on rates	0.12 and 0.2
horizVel = 3.0~3.01	Will move somewhat faster	63 seconds	Yes, moves fast enough to prevent errant movement while giving enough time for response	3.01
Natfreqheight = 3.0-5	Faster response to yaw, better performance, then reduce damping	72 seconds, lower values couldn't consistently be better than 72 seconds	Yes, response seems optimal to keep drone in the room	4.0 3.0

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

	amount to allow for more stable conditions			
Damping ratio = 0.707	Ideal underdamped 2nd order condition	Seems like it's better	Yes	0.707

7. Provide the full listing of your MainLoop function, and provide any other functions you introduced. Make sure that everything is well commented. [5 %]

```
#include "UserCode.hpp"
#include "UtilityFunctions.hpp"
#include "Vec3f.hpp"

#include <stdio.h> //for printf

//An example of a variable that persists beyond the function call.
float exampleVariable_float = 0.0f;  //Note the trailing 'f' in the number. This is to force single precision floating point.
Vec3f exampleVariable_Vec3f = Vec3f(0, 0, 0);
int exampleVariable_int = 0;

//We keep the last inputs and outputs around for debugging:
MainLoopInput lastMainLoopInputs;
MainLoopOutput lastMainLoopOutputs;

//Some constants that we may use:
const float mass = 32e-3f;  // mass of the quadcopter [kg]
const float gravity = 9.81f;  // acceleration of gravity [m/s^2]
const float inertia_xx = 16e-6f;  //MMOI about x axis [kg.m^2]
const float inertia_yy = inertia_xx;  //MMOI about y axis [kg.m^2]
const float inertia_zz = 29e-6f;  //MMOI about z axis [kg.m^2]

const float dt = 1.0f / 500.0f;  //[s] period between successive calls to MainLoop


// NEW CODE STARTS HERE

Vec3f estGyroBias = Vec3f(0,0,0) ;
```

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

```
float estRoll = 0;
float estPitch = 0;
float estYaw = 0;
float kappa=0.01;
float l = 33e-3f;
float desPitchAngle =0;
float estHeight=0;
float estVelocity_1 = 0;
float estVelocity_2 = 0;
float estVelocity_3 = 0;
float lastHeightMeas_meas = 0;
float lastHeightMeas_time = 0;

float estPos_1 = 0.0f; // Added Positional Estimator
float estPos_2 = 0.0f; // Added Positional Estimator
const float desPos_1 = 0.0f; // Desired position which should be constant zero
const float desPos_2 = 0.0f; // Desired position which should be constant zero


MainLoopOutput MainLoop(MainLoopInput const &in) {
  //Your code goes here!
  // The function input (named "in") is a struct of type
  // "MainLoopInput". You can understand what values it
  // contains by going to its definition (click on "MainLoopInput",
  // and then hit <F3> -- this should take you to the definition).
  // For example, "in.userInput.buttonBlue" is true if the
  // blue button is pushed, false otherwise.

  //Define the output numbers (in the struct outVals):
  MainLoopOutput outVals;
//  motorCommand1 -> located at body +x +y
//  motorCommand2 -> located at body +x -y
//  motorCommand3 -> located at body -x -y
//  motorCommand4 -> located at body -x +y


              // ADDITIONAL CODE BEGINNING
  float const mixHorizVel = 0.1f;
  float const mixHeight = 0.3f;
  float p = 0.01;  //ATTITUDE ESTIMATOR MIXING


  if (in.currentTime < 1.0f ) {
  estGyroBias = estGyroBias + (in.imuMeasurement.rateGyro / 500.0f) ;
```

```
}
Vec3f rateGyro_corr = in.imuMeasurement.rateGyro - estGyroBias ; //Bias corrected Rate Gyro Measurement
estRoll=(1-p)*(estRoll+dt*rateGyro_corr.x)+p*in.imuMeasurement.accelerometer.y/gravity;
estPitch=(1-p)*(estPitch+dt*rateGyro_corr.y)-p*in.imuMeasurement.accelerometer.x/gravity;
estYaw=estYaw+dt*rateGyro_corr.z;  //ESTIMATED ANGLES

//height estimator:
//prediction step:
estHeight = estHeight + estVelocity_3 * dt;
estVelocity_3 = estVelocity_3 + 0*dt;  //CHANGEABLE
// Correction step, directly after the prediction step:

if (in.heightSensor.updated) {
    // Check that the measurement is reasonable
    if (in.heightSensor.value < 5.0f) {
        float hMeas = in.heightSensor.value * cosf(estRoll) * cosf(estPitch);
        estHeight = (1 - mixHeight) * estHeight + mixHeight * hMeas;
        float v3Meas = (hMeas - lastHeightMeas_meas) / (in.currentTime - lastHeightMeas_time);
        estVelocity_3 = (1 - mixHeight) * estVelocity_3 + mixHeight * v3Meas;

        // Store this measurement for the next velocity update
        lastHeightMeas_meas = hMeas;
        lastHeightMeas_time = in.currentTime;
    }
}


//Horizontal Estimator
//Prediction
estVelocity_1 = estVelocity_1 + 0*dt; //CHANGEABLE
estVelocity_2 = estVelocity_2 + 0*dt; //CHANGEABLE
// Correction step:
if (in.opticalFlowSensor.updated) {
    float sigma_1 = in.opticalFlowSensor.value_x;
    float sigma_2 = in.opticalFlowSensor.value_y;

    float div = (cosf(estRoll) * cosf(estPitch));
    if (div > 0.5f) {  //CHANGEABLE LOTS OF EFFORT NEEDED
        float deltaPredict = estHeight / div; // This is delta in the equation
        float v1Meas = (-sigma_1 + in.imuMeasurement.rateGyro.y) * deltaPredict;
        float v2Meas = (-sigma_2 - in.imuMeasurement.rateGyro.x) * deltaPredict;

        estVelocity_1 = (1 - mixHorizVel) * estVelocity_1 + mixHorizVel * v1Meas;
        estVelocity_2 = (1 - mixHorizVel) * estVelocity_2 + mixHorizVel * v2Meas;
```

```
    }
  }


//TIME CONSTANTS ALL VARIABLE
  float const timeConstant_rollRate = 0.015f; // [ s ]
  float const timeConstant_pitchRate = timeConstant_rollRate; // [ s ]
  float const timeConstant_yawRate = 0.065f; // [ s ]
  float const timeConstant_rollAngle = 0.12f; // [ s ]
  float const timeConstant_pitchAngle = timeConstant_rollAngle; // TIME CONSTANTS
  float const timeConstant_yawAngle = 0.2f; // [ s ]

  //VERTICAL ESTIMATOR SECOND ORDER
  const float natFreq_height = 4.0f;
  const float dampingRatio_height = 0.707f;
const float timeConst_horizVel = 3.01f;
const float timeConst_position = 12.0f; // Added Positional Time Constant

//ADDED POSITIONAL CONTROL
estPos_1 = estPos_1 + estVelocity_1 * dt; // Integration of Velocity for estimator
estPos_2 = estPos_2 + estVelocity_2 * dt; // Integration of Velocity for estimator
float posError_1 = desPos_1 - estPos_1; // Position error in X
float posError_2 = desPos_2 - estPos_2; // Position error in Y



  //Horizontal Control
  float desAcc1 = -(1 / timeConst_horizVel) * estVelocity_1 - (1 / timeConst_position) * posError_1; //CHANGED
FOR POS CONTROL
  float desAcc2 = -(1 / timeConst_horizVel) * estVelocity_2 - (1 / timeConst_position) * posError_2; //CHANGED
FOR POS CONTROL
  float desRoll = -desAcc2/gravity;
  float desPitch = desAcc1/gravity;
  float desYaw = 0;
  Vec3f thetaDes = Vec3f(desRoll,desPitch,desYaw);



  //Attitude Estimator
  float pcmd= -(estRoll-thetaDes[0])/timeConstant_rollAngle;
  float qcmd= -(estPitch-thetaDes[1])/timeConstant_pitchAngle;
  float rcmd= -(estYaw-thetaDes[2])/timeConstant_yawAngle; //ANGULAR VELOCITY FROM ANGLE
DIFFERENCE
```

Noah Koch, Jason Khatkar, Ryan Montis
ME136, 11/13/2024

```
  Vec3f cmdAngVel = Vec3f(pcmd, qcmd, rcmd);
  float pdotcmd= -(rateGyro_corr[0]-cmdAngVel[0])/timeConstant_rollRate;
  float qdotcmd= -(rateGyro_corr[1]-cmdAngVel[1])/timeConstant_pitchRate;
  float rdotcmd= -(rateGyro_corr[2]-cmdAngVel[2])/timeConstant_yawRate; //ANGULAR ACCELERATION
FROM ANG VELOCITY
  Vec3f cmdAngAcc = Vec3f(pdotcmd,qdotcmd, rdotcmd);

  //Vertical Height Estimator
  const float desHeight = 0.5f;
  const float desAcc3 = -2 * dampingRatio_height * natFreq_height * estVelocity_3 - natFreq_height *
natFreq_height * (estHeight - desHeight);
  float desNormalizedAcceleration=(gravity+desAcc3)/(cosf(estRoll)*cosf(estPitch));


  //Lab 4
  float csum= mass*desNormalizedAcceleration; //Insert the sum force required
  float n1=cmdAngAcc[0]*inertia_xx; //Torque 1
  float n2 =cmdAngAcc[1]*inertia_yy; //Torque 2
  float n3 =cmdAngAcc[2]*inertia_zz; //Torque 3
// MOTOR FORCE MIXER BEGIN
  float motorf1 = 0.25*(csum + n1/l - n2/l + n3/kappa);
  float motorf2 = 0.25*(csum - n1/l - n2/l - n3/kappa);
  float motorf3 = 0.25*(csum - n1/l + n2/l + n3/kappa);
  float motorf4 = 0.25*(csum + n1/l + n2/l - n3/kappa);
 // MOTOR FORCE MIXER END
//Lab 2
  float reqRad1= speedFromForce(motorf1); //Get a value for the required speed
  float reqRad2= speedFromForce(motorf2); //Get a value for the required speed
  float reqRad3= speedFromForce(motorf3); //Get a value for the required speed
  float reqRad4= speedFromForce(motorf4); //Get a value for the required speed
  int reqPWM1 = pwmCommandFromSpeed(reqRad1); //Get a value for the required PWM needed
  int reqPWM2 = pwmCommandFromSpeed(reqRad2); //Get a value for the required PWM needed
  int reqPWM3 = pwmCommandFromSpeed(reqRad3); //Get a value for the required PWM needed
  int reqPWM4 = pwmCommandFromSpeed(reqRad4); //Get a value for the required PWM needed



   if (in.userInput.buttonBlue == true) { // If the blue button is pressed down
     outVals.motorCommand1 = reqPWM1;
     outVals.motorCommand2 = reqPWM2;
     outVals.motorCommand3 = reqPWM3; //Set all four motors to the required PWM value
     outVals.motorCommand4 = reqPWM4;
   }
     else { // When not pushed down
```

```
    outVals.motorCommand1 = 0;
    outVals.motorCommand2 = 0;
    outVals.motorCommand3 = 0; //Set all four motors to 0
    outVals.motorCommand4 = 0;
    desPitchAngle=0;
    }

   outVals.telemetryOutputs_plusMinus100[0] = estRoll;
   outVals.telemetryOutputs_plusMinus100[1] = estPitch;
   outVals.telemetryOutputs_plusMinus100[2] = estYaw;
   outVals.telemetryOutputs_plusMinus100[3] = estVelocity_1;
   outVals.telemetryOutputs_plusMinus100[4] = estVelocity_2;
   outVals.telemetryOutputs_plusMinus100[5] = estVelocity_3;
   outVals.telemetryOutputs_plusMinus100[6] = estHeight;
   outVals.telemetryOutputs_plusMinus100[7] = desRoll;
   outVals.telemetryOutputs_plusMinus100[8] = desPitch;
   outVals.telemetryOutputs_plusMinus100[9] = desNormalizedAcceleration;
   outVals.telemetryOutputs_plusMinus100[10] = posError_1;
   outVals.telemetryOutputs_plusMinus100[11] = posError_2;




  //copy the inputs and outputs:
  lastMainLoopInputs = in;
  lastMainLoopOutputs = outVals;
  return outVals;
}

void PrintStatus() {
  //For a quick reference on the printf function, see: http://www.cplusplus.com/reference/cstdio/printf/
  // Note that \n is a "new line" character.
  // Also, note that to print a `float` variable, you have to explicitly cast it to
  //  `double` in the printf function, and explicitly specify precision using something
  //  like %6.3f (six significant digits, three after the period). Example:
  //   printf("  exampleVariable_float = %6.3f\n", double(exampleVariable_float));

  //Accelerometer measurement
  printf("Acc:");
  printf("x=%6.3f, ",
        double(lastMainLoopInputs.imuMeasurement.accelerometer.x));
  printf("y=%6.3f, ",
        double(lastMainLoopInputs.imuMeasurement.accelerometer.y));
  printf("z=%6.3f, ",
```

```
        double(lastMainLoopInputs.imuMeasurement.accelerometer.z));
  //Rate gyro measurement
  printf("Gyro:");
  printf("x=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.x));
  printf("y=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.y));
  printf("z=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.z));
  printf("\n");
  //Rate gyro minus bias measurement
  printf("Gyro minus bias:");
  printf("x=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.x - estGyroBias.x));
  printf("y=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.y - estGyroBias.y));
  printf("z=%6.3f, ", double(lastMainLoopInputs.imuMeasurement.rateGyro.z - estGyroBias.z));
  printf("\n");
  printf("Estimated Attitude:");
  printf("x=%6.3f, ", double(estRoll));
  printf("y=%6.3f, ", double(estPitch));
  printf("z=%6.3f, ", double(estYaw));
  printf("\n");

  printf("Last range = %6.3fm, ", \
        double(lastMainLoopInputs.heightSensor.value)) ;
  printf("Last flow: x=%6.3f, y=%6.3f\n", \
        double(lastMainLoopInputs.opticalFlowSensor.value_x) , \
        double(lastMainLoopInputs.opticalFlowSensor.value_y));


  printf("Example variable values:\n");
  printf("  exampleVariable_int = %d\n", exampleVariable_int);
  //Note that it is somewhat annoying to print float variables.
  //  We need to cast the variable as double, and we need to specify
  //  the number of digits we want (if you used simply "%f", it would
  //  truncate to an integer.
  //  Here, we print 6 digits, with three digits after the period.
  printf("  exampleVariable_float = %6.3f\n", double(exampleVariable_float));

  //We print the Vec3f by printing it's three components independently:
  printf("  exampleVariable_Vec3f = (%6.3f, %6.3f, %6.3f)\n",
        double(exampleVariable_Vec3f.x), double(exampleVariable_Vec3f.y),
        double(exampleVariable_Vec3f.z));

  //just an example of how we would inspect the last main loop inputs and outputs:
  printf("Last main loop inputs:\n");
  printf("  batt voltage = %6.3f\n",
        double(lastMainLoopInputs.batteryVoltage.value));
```

```
  printf("  JS buttons: ");
  if (lastMainLoopInputs.userInput.buttonRed)
    printf("buttonRed ");
  if (lastMainLoopInputs.userInput.buttonGreen)
    printf("buttonGreen ");
  if (lastMainLoopInputs.userInput.buttonBlue)
    printf("buttonBlue ");
  if (lastMainLoopInputs.userInput.buttonYellow)
    printf("buttonYellow ");
  if (lastMainLoopInputs.userInput.buttonArm)
    printf("buttonArm ");
  printf("\n");
  printf("Last main loop outputs:\n");
  printf("  motor commands: = %6.3f\t%6.3f\t%6.3f\t%6.3f\t\n",
        double(lastMainLoopOutputs.motorCommand1),
        double(lastMainLoopOutputs.motorCommand2),
        double(lastMainLoopOutputs.motorCommand3),
        double(lastMainLoopOutputs.motorCommand4));
}
```